

Are Agent Memory Systems Worth It for Coding Agents?

MemBench: A Contamination-Controlled Benchmark Pilot with Pinned Open-Weight Models

MemBench Project*

July 3, 2026

Abstract

Persistent memory systems for LLM coding agents are widely marketed but weakly evidenced: every prominent published comparison is vendor-produced, and the two head-to-head disputes on record (Zep vs. Mem0 on LoCoMo) show that configuration choices alone swing reported scores by more than 25 percentage points. We present **MemBench**, an executable, contamination-controlled benchmark harness for measuring whether agent memory earns its cost on coding tasks, and report a pilot study comparing **seven memory arms** — no-memory, raw-history RAG, structured project memory, a claude-mem-style FTS5 pipeline, Mem0 OSS, Graphiti (FalkorDB), and graphify — under a single agent scaffold (Claude Code) driven end-to-end by a *pinned open-weight model* (Qwen3-Coder-30B via vLLM), with extraction and embedding for every memory system routed to the same local endpoints. On our five-task pilot suite, *every arm resolved every task* (35/35, independently re-verified). The pilot’s chief findings are methodological: (i) the agent scaffold dominates the model — the same model scored 0/6 single-shot but 100% agentic; (ii) un-isolated harnesses silently corrupt results — before environment isolation, headless agents inherited user-level tools and auto-memory, token costs spread $2.4\times$ across arms, and the platform systems appeared most expensive; after isolation the spread collapsed to $1.36\times$ and the cost ranking *inverted*, with Mem0 and graphify cheapest; and (iii) memory’s first measurable effect is on agent *behavior and cost*, not correctness. We release the harness, tasks, per-turn trajectories, and all failure modes as a foundation for the statistically powered evaluation the field currently lacks.

1 Introduction

Memory layers for coding agents promise cross-session learning: remember the project’s conventions, past bugs, and setup quirks, and the agent should resolve issues faster and cheaper. The market took the promise seriously — Mem0 raised \$24M, Zep commercialized Graphiti, and the most popular Claude Code memory plugin (claude-mem) exceeds 85k GitHub stars. The evidence did not keep pace:

- **Vendor numbers do not survive audit.** Zep reported 84% on LoCoMo; a corrected third-party evaluation measured 58.4%. Mem0’s headline +26% is measured on a benchmark whose answer key contains 6.4% errors and whose LLM judge accepts 62.8% of intentionally wrong answers [2].

*Benchmark code, dataset, configurations, and full agent trajectories at the MemBench repository. Inference: Qwen3-Coder-30B (AWQ 4-bit) served by vLLM 0.24.0 on a single RTX 5090.

- **Simple baselines embarrass platforms.** A plain filesystem-plus-grep agent beat Mem0’s graph variant on Letta’s own benchmark (74.0 vs. 68.5) [3]; on MemoryAgentBench, Mem0 scored 3.4% on test-time learning against 87.6% for a long-context baseline [4].
- **The standard coding benchmark is compromised.** OpenAI’s 2026 audit of SWE-bench Verified found 59.4% flawed tests among consistent failures and verbatim gold-patch memorization across frontier models [1], making “memory lift” on it indistinguishable from contamination recall.

No coding-agent memory benchmark with executable oracles, contamination controls, pinned open-weight models, and statistical power exists. MemBench is our attempt to build one. This paper reports the design and a *pilot*: a deliberately small, fully instrumented dry run whose purpose is to validate the harness end-to-end and surface the failure modes that would corrupt a larger study. We are explicit about scale: five tasks, one seed, one model — the pilot answers “does the methodology work?”, not yet “which memory system wins?”.

2 Related Work

Memory systems. Mem0 (vector store with LLM fact extraction; the 2026 OSS release replaced the published two-phase pipeline with single-pass ADD-only extraction), Graphiti/Zep (bi-temporal knowledge graph, hybrid semantic+BM25+graph retrieval), Letta/MemGPT (self-edited memory blocks), Cogniee (ECL knowledge-graph pipeline), claude-mem (Claude Code plugin: hook-captured observations in SQLite FTS5), graphify (repository knowledge graphs via tree-sitter + LLM semantic extraction), and Anthropic’s native memory tool and auto-memory. Of these, only ReasoningBank-style distilled episodic memory has a measured lift on a repository-level coding benchmark (+4.6pp on SWE-bench Verified, with the contamination caveat above) [5].

Benchmarks. LoCoMo and LongMemEval (conversational recall; LoCoMo demonstrably unreliable), MemoryAgentBench (four memory competencies; conversational), EvoMemBench (memory value shrinks as context windows grow), STALE (best frontier model detects invalidated memories only 55.2% of the time), and SWE Context Bench (319 interdependent GitHub tasks; oracle summary reuse +8pp) — the closest prior work to MemBench, but single-run and without cost accounting. For contamination-safe task sourcing we build on SWE-bench-Live (post-2024 issues, monthly refresh) and the SWE-bench harness pattern of Dockerized per-instance environments with fail-to-pass/pass-to-pass oracles [6].

Measurement hazards. Run-to-run pass@1 on SWE-bench-style evals varies 2.2–6.0pp across identical reruns — the same order as typical memory effect sizes — so single-run comparisons are uninterpretable [7]. Our companion design document additionally identifies a *carry-forward hazard* specific to memory evaluation: in 70% of consecutive Django task pairs, the earlier task’s fix is already present in the later task’s base checkout, inflating naive “memory lift”.

3 MemBench Design

3.1 Task model

Each instance models an agent returning to a project: an issue, a workspace snapshot (template repository), an executable oracle (pytest fail-to-pass and pass-to-pass sets, with test paths protected from agent modification), and a time-split *memory corpus* — structured project memory, event history, and documents, all strictly predating the task. A pre-check runs the fail-to-pass tests

before the agent starts; an instance only counts as resolved if the bug demonstrably existed, the fail-to-pass tests pass afterward, and no pass-to-pass test regresses.

3.2 Pilot task suite

Five small Python tasks instantiate the design’s memory-need taxonomy:

Instance	Category	Description
demo__001	M2 episodic	Decimal division leaks backend exception
demo__002	M2 repeated pattern	Same leak class, different module; past fix in corpus
demo__003	M3 convention	Legacy config aliases; convention only in memory
demo__004	S stale trap	Corpus contains an invalidated module-location note
demo__005	M0 + distractors	Trivial off-by-one; corpus is all distractors

3.3 Memory arms

All arms share one scaffold — Claude Code in headless mode — and differ only in what the memory layer injects into the task prompt. Retrieved items are capped at 2,000 tokens.

Arm	System	Storage / retrieval	Extraction LLM
none	—	—	—
raw_rag	lexical RAG	corpus JSONL, lexical scoring	none
structured	typed project memory	curated facts, lexical	none
claude__mem	claude-mem pipeline replica	SQLite FTS5, BM25	none (corpus pre-structured)
mem0	Mem0 OSS 2.0.11	Qdrant (local), vector	qwen3-coder-30b (vLLM provider)
graphiti	Graphiti 0.29.x	FalkorDB temporal graph, hybrid	qwen3-coder-30b + qwen3-embedding
graphify	graphify (safishamsi)	knowledge graph, BFS query	qwen3-coder-30b (OpenAI-compatible)

The claude-mem arm replicates the plugin’s storage and search layer (FTS5, BM25 ranking) rather than running the shipped plugin: the installed release cannot re-point its extraction step at a local model, and running it against a commercial API would break the pinned-model control. Letta and Cognee are excluded from the pilot: Letta’s Claude Code bridge requires its cloud service, and Cognee’s documentation recommends larger extraction models than our pinned backbone.

3.4 Pinned open-weight stack

Reproducibility requires that no arm silently depends on a commercial API. The agent, and the extraction/embedding calls of every memory system, run against two local endpoints: Qwen3-Coder-30B-A3B-Instruct (AWQ 4-bit) and Qwen3-Embedding-0.6B, both served by vLLM 0.24.0 on one RTX 5090 (32 GB). vLLM’s native Anthropic-compatible `/v1/messages` endpoint lets Claude Code drive the open-weight model directly; tool calling requires `--enable-auto-tool-choice --tool-call-parser qwen3_coder`.

3.5 Metrics

Per instance and arm: resolution (executable oracle only — no LLM judges), wall time, API turns, per-tool call counts (from full stream-JSON trajectories), total tokens, retrieved memory items, and memory-store construction time. Cost for the pinned stack is electricity; token counts are the portable cost signal.

4 Harness Findings: What Silently Corrupts Memory Benchmarks

The pilot’s most transferable results are five failure modes we hit, diagnosed from trajectories, and fixed. Any of them would have invalidated a larger study.

1. **Scaffold dominates model.** Single-shot patch generation with Qwen3-Coder-30B resolved 0/6 attempts (it cannot see the repository); the same model inside the agentic scaffold resolved every pilot task. Memory comparisons made on a crippled scaffold measure nothing about memory.
2. **Headless agents never compact.** Claude Code assumes a 200k context for unknown models and disables auto-compaction in print mode; agents spiraled to 70+ turns and died at the 65k serving limit (~ 7 minutes and 3.4M input tokens per failure). Fixes: declare the real window (`CLAUDE_CODE_MAX_CONTEXT_TOKENS`), cap output tokens, and impose an explicit `--max-turns` budget — itself a benchmark parameter.
3. **Environment contamination.** Un-isolated headless runs inherit the host user’s Claude Code configuration: task-management tools, installed skills, third-party memory plugins (including `claude-mem`’s capture hooks), and auto-memory, which wrote `MEMORY.md` files that appeared *inside model patches*. Trajectories showed agents spending 60–80% of capped turns on inherited task tools after the fix had landed. Benchmark runs must use an isolated configuration directory with auto-memory disabled.
4. **Bounded failure is not harness error.** Hitting a turn budget must be recorded as a scoreable unresolved-or-resolved attempt (the oracle decides), not discarded: in *every* capped pilot run the fix had already landed by turn ~ 6 –12, so discarding capped runs would have systematically undercounted resolutions.
5. **Memory-platform pinning is fragile in practice.** Pinning all systems to one local stack surfaced: an embedding-dimension parameter rejected by vLLM (Mem0), a search-API signature change between the documented and shipped versions (Mem0 2.x), a global telemetry store that deadlocks concurrent corpora (Mem0), and a deprecated default graph backend that creates no indexes at all (Graphiti/Kuzu; we substituted the supported FalkorDB). None of these appear in any vendor benchmark report.

5 Pilot Results

5.1 Resolution and verification

All seven arms resolved all five tasks (Table 1). Every resolution was verified outside the harness: the recorded source patch was applied to a pristine template checkout and the oracle re-run (35/35 re-pass). Oracle prechecks confirmed each bug failed before the agent ran, and trajectory analysis confirmed the fix landed well inside the turn budget in every capped run.

5.2 Isolation flipped the cost ranking

The identical matrix run *without* environment isolation produced a different picture: token costs spread $2.4\times$ (561k–1.36M per task), the platform arms (Mem0, Graphiti) appeared most expensive and hit the turn cap most often, and trajectories showed agents spending most post-fix turns on tools inherited from the host user’s environment. After isolation, the spread collapsed to $1.36\times$

Table 1: Isolated pilot run: 7 memory arms $\times$ 5 tasks, Qwen3-Coder-30B driving Claude Code (30-turn budget). *Re-verified* = recorded patch re-applied to a pristine checkout re-passes the oracle. *Capped* = runs hitting the turn budget (all still resolved). Averages over the arm’s 5 runs.

Arm	Resolved	Re-verified	Capped	Wall (s)	Turns	Tokens	Mem items
graphify	5/5	5/5	2	36	20.4	460,143	1.0
mem0	5/5	5/5	2	36	20.8	480,847	8.4
raw_rag	5/5	5/5	3	48	21.8	504,060	1.8
none	5/5	5/5	3	44	23.0	522,292	0.0
structured	5/5	5/5	3	38	23.8	540,325	1.4
graphiti	5/5	5/5	3	39	24.0	547,330	5.0
claude_mem	5/5	5/5	4	51	27.0	624,544	3.8

(460k–625k) and the ranking inverted — Mem0 and graphify became the *cheapest* arms. The pre-isolation “platform bloat” was largely contamination, not the platforms. We consider this the pilot’s most important quantitative result: an un-isolated harness did not merely add noise, it *systematically reordered* the systems under test.

5.3 What cost differences survive isolation

The surviving spread is modest and, at $n=1$ per cell, indistinguishable from run-to-run variance (arms swung 0/5–5/5 across reruns during harness debugging when infrastructure faults intervened). Two qualitative observations still hold across all runs: injected memory count is uncorrelated with success on tasks this easy (Mem0’s 8.4 items and none’s 0 both score 5/5), and the distractor-corpus task (M0) shows memory arms paying for irrelevant context the no-memory arm never sees — the overhead channel exists even when it does not change outcomes.

5.4 Turn-budget behavior

2–4 runs per arm hit the 30-turn budget, yet every capped run had already produced the fix (typically by turn 6–12); remaining turns went to redundant verification and task-management churn. The budget therefore acted as a pure cost control on this suite — it changed no outcomes — but it is a benchmark parameter that harder tasks will interact with, and must be reported.

6 Discussion: Are Memory Systems Worth It?

The pilot cannot answer the headline question — its tasks are too easy (the no-memory arm scores 100%) and $n=1$ per cell sits inside known run-to-run variance. What it does establish:

- **Harness hygiene is worth more percentage points than memory system choice.** Environment isolation moved per-arm token costs by up to $2.8\times$ and reordered the systems under test; no published memory comparison we are aware of reports its isolation regime. Until a benchmark controls scaffold, budget, and environment, its ranking of memory systems is not measuring memory systems.
- **On tasks the agent can solve alone, memory is pure overhead — but small.** With correctness saturated, the isolated cost spread across arms was $1.36\times$; memory construction (milliseconds for FTS, seconds for graph extraction, longest for Mem0’s per-item LLM pipeline) remains the larger practical difference at corpus scale.

- **Memory’s first measurable effect is behavioral.** Injected context changed how long agents ran and what they did after fixing the bug, before it changed whether they fixed it. Cost and behavior metrics must be first-class in any memory evaluation.
- **A fair platform comparison on pinned open weights is feasible** — all four integrated systems ran extraction and embedding on local endpoints — but only after fixing failure modes that vendor-run evaluations have no incentive to report (Section 4).

7 Limitations and Next Steps

Five synthetic tasks, one seed, one ordering, one model, no memory-write evaluation, and a corpus small enough that construction-time differences (milliseconds to minutes) do not yet bind. The funded next step is the designed pilot: 100–150 SWE-bench-Live-derived tasks in dependency-mined sequences with carry-forward annotation, 4–5 seeds $\times$ 3 orderings, paired McNemar and clustered-bootstrap analysis, stale/distractor/poison splits, and the decision rule pre-registered in the design document: if simple structured memory is within noise of the platforms, the recommendation is files and grep.

Reproducibility

All configurations, task templates, memory corpora, adapter implementations, run outputs, and full per-turn trajectories (stream-JSON, including every tool call) accompany the repository. The serving stack is two pinned open-weight checkpoints on one consumer GPU; total pilot cost was local electricity.

References

- [1] OpenAI. *Why we no longer evaluate SWE-bench Verified*. Feb 23, 2026.
- [2] Mem0 vs. Zep LoCoMo dispute: corrected evaluation, GitHub issue thread, May 2025; Zep, *Lies, Damn Lies, & Statistics*, 2025.
- [3] Letta. *Benchmarking AI agent memory: is a filesystem all you need?* 2025.
- [4] Hu et al. *MemoryAgentBench*. ICLR 2026. arXiv:2507.05257.
- [5] *ReasoningBank*. arXiv:2509.25140.
- [6] *SWE-bench-Live*. arXiv:2505.23419.
- [7] Miller et al. *Adding Error Bars to Evals*. arXiv:2411.00640.
- [8] *STALE: Testing memory invalidation*. arXiv:2605.06527.
- [9] *SWE Context Bench*. arXiv:2602.08316.